

CS3002E Compiler Design

Monsoon 2025

Lecture #22

Semantic Analysis - Type Expressions, Type checking, Type
Conversions

Department of Computer Science & Engineering
NIT Calicut

Type Checking

What are the type checks done by the compiler for the following assignment in a C language program?

```
a = b+c;
```

Type Checking

What are the type checks done by the compiler for the following assignment in a C language program?

```
x = f (y);
```

Types

- What is the information conveyed by the following declaration?

```
int a; float b;
```

Types

- What is the information conveyed by the following declaration?

```
int a; float b;
```

- Type of variable a is int. What is meant by **data type** ?

Type

Type:

- a set of values (domain of values).

- a set of operations on these values.

Type Checking

What are the type checks done by the compiler for each of the following assignments in a C language program?

```
a = b+c;
```

```
x = f (y);
```

Can we assign type for expression/statement?

Type Checking

Type Checking: checks if the types of operands match the type expected by an operator.

Type System

A set of types and a set of typing rules to assign types to various language constructs.

Types

For the following C code fragment, indicate the type of the identifiers, a, b, c, f, and s:

```
int a; float b;  
int c[10];  
int f( int x, float y);  
struct student{int num; float marks;} s;
```

Type Expression

What is the type of `a` in `int a[10]`?

Type Expression : examples

`array (10, int)`: an array of size 10, base type is *int*

Type Expression

Given:

```
boolean f (int x);
```

What is the type of f ?

Type Expression : examples

- `array (10, int)`: an array of size 10, base type is *int*
- `int → boolean`: a function that takes an *int* type argument and returns a *boolean* value

Type Expression : examples

- $\text{array}(10, \text{int})$: an array of size 10, base type is *int*
- $\text{int} \rightarrow \text{boolean}$: a function that takes an *int* type argument and returns a *boolean* value
- $\text{int} \times \text{float} \rightarrow \text{boolean}$: a function that takes two arguments (an *int* and a *float*) and returns a *boolean* value

Type Expression

A *type expression* is used to denote the type of a language construct. A type expression can be:

- a basic type e.g. *integer*, *boolean*, *char*, *float*, *void* (denotes absence of value), *type_error* (to signal error during type checking)
- a type name
- a type constructor applied to appropriate arguments

Type expression may contain variables whose values are type expressions.

Type Expression: Type Constructor

- $array(l, t)$: the type constructor *array* applied to a number l and a type expression t
- $s \rightarrow t$: the type constructor $\rightarrow$ applied to type expressions s and t , denoting the type of functions with argument type s and return type t
- $s \times t$: the type constructor $\times$ applied to type expressions s and t , denoting list/tuple of types
- $record((l_1 \times t_1) \times (l_2 \times t_2))$: the type constructor *record* applied to a list of $(label, type)$ pairs, denoting the type of a record
- $pointer(t)$: the type constructor *pointer* applied to a type t , denoting the type of a pointer pointing to an object of type t

Exercise

Suppose `f` is declared as `int f(int x);` Write a type expression denoting the type of `f`.

- **Data Type**: a set of values plus a set of operations on those values. e.g. set of integers with integer arithmetic operations - addition, subtraction etc.
- **Type system** of a language: set of types and a collection of logical rules to assign types to various language constructs. Type system is part of the language specification.
- **Typechecker**: an implementation of the type system. Type-checker assigns a *type expression* to each component of the source program and checks if these type expressions conform to the type system for the source language.

Type System for a sample language

Production	Semantic Rules
$E \rightarrow \mathbf{num}$	$E.type = int$
$E \rightarrow \mathbf{id}$	$E.type = getType(\mathbf{id}.entry)$
$E \rightarrow E_1 + E_2$	$if (E_1.type = int \text{ and } E_2.type = int)$ $then E.type = int \text{ else } E.type = type_error$
...	...

$getType(\mathbf{id}.entry)$ is assumed to retrieve the type of **id** from the Symbol Table and return it.

Type System for a sample language

Production	Semantic Rules
$E \rightarrow \mathbf{num}$	$E.type = int$
$E \rightarrow \mathbf{id}$	$E.type = getType(\mathbf{id}.entry)$
$E \rightarrow E_1 + E_2$	<i>if ($E_1.type = int$ and $E_2.type = int$) then $E.type = int$ else $E.type = type_error$</i>
$E \rightarrow E_1[E_2]$	...

Type System for a sample language

Production	Semantic Rules
$E \rightarrow \mathbf{num}$	$E.type = int$
$E \rightarrow \mathbf{id}$	$E.type = getType(\mathbf{id}.entry)$
$E \rightarrow E_1 + E_2$	$if (E_1.type = int \text{ and } E_2.type = int)$ $then E.type = int \text{ else } E.type = type_error$
$E \rightarrow E_1[E_2]$	$if (E_2.type = int \text{ and } E_1.type = array(s, t))$ $then E.type = t \text{ else } E.type = type_error$
$E \rightarrow E_1(E_2)$	...

Type System for a sample language

Production	Semantic Rules
$E \rightarrow \mathbf{num}$	$E.type = int$
$E \rightarrow \mathbf{id}$	$E.type = getType(\mathbf{id}.entry)$
$E \rightarrow E_1 + E_2$	$if (E_1.type = int \text{ and } E_2.type = int)$ $then E.type = int \text{ else } E.type = type_error$
$E \rightarrow E_1[E_2]$	$if (E_2.type = int \text{ and } E_1.type = array(s, t))$ $then E.type = t \text{ else } E.type = type_error$
$E \rightarrow E_1(E_2)$	$if (E_2.type = s \text{ and } E_1.type = s \rightarrow t) \text{ then}$ $E.type = t \text{ else } E.type = type_error$
...	...

Type System for a sample language

Production	Semantic Rules
$S \rightarrow \mathbf{id} = E$	<i>if ($E.type = getType(id.entry)$) then $S.type = void$ else $S.type = type_error$</i>
$S \rightarrow \mathbf{if}(E) \mathbf{then} S_1$	<i>if ($E.type = bool$) then $S.type = S_1.type$ else $S.type = type_error$</i>
$S \rightarrow \mathbf{while}(E) S_1$	<i>if ($E.type = bool$) then $S.type = S_1.type$ else $S.type = type_error$</i>

Type Conversions

Conversion from one type to another

- *Implicit*: done automatically by the compiler (known as *coercions*). Type Checker inserts required conversion operations into the intermediate representation.
e.g. given an expression for adding an *int* and a *float*, compiler inserts code to convert the *int* operand to *float* type.
- *Explicit*: conversions written by the programmer (known as *casts*).

Type Checking rules for coercion from int to float

Production	Semantic Rules
$E \rightarrow E_1 + E_2$	<i>if ($E_1.type = int$ and $E_2.type = int$) then $E.type = int$ else if ($E_1.type = int$ and $E_2.type = float$) then $E.type = float$ else if ($E_1.type = float$ and $E_2.type = int$) then $E.type = float$ else if ($E_1.type = float$ and $E_2.type = float$) then $E.type = float$ else $E.type = type_error$</i>

Type Equivalence

Structural equivalence of type expressions: the expressions are either the same basic type or are formed by applying the same type constructor to structurally equivalent types.

Quiz

1. Suppose `f` is declared as `int f(int x);` Write a type expression denoting the type of `f`.
2. Suppose `a` is declared as `int a[10][5];` Write a type expression denoting the type of `a`.
3. What is the domain of values for type `void`?

References

References:

- Aho A.V., Lam M.S., Sethi R., and Ullman J.D. Compilers: Principles, Techniques, and Tools (ALSU). Pearson Education, 2007¹.

Further reading:

- ALSU Chapter 6 - sections 6.5 - 6.5.1, 6.5.2 (only the relevant portions)

¹some of the topics are from another edition by Aho, Sethi and Ullman